

Project 1 — Cloud-Native Nutritional

Insights: Master Overview

GitHub: <https://github.com/YUANDONG-YANG/cloud-computing/tree/main/Project1>

CPSY 300 Cloud Computing · Fall 2026 · Team: Yuandong Yang, Ethan Bayarsaikhan, Justin Norman-Rance · Review date: 2026-09-30

Analyse the course `All_Diets.csv` (7,806 recipes, 5 diets) with Pandas, containerise it with Docker, simulate Azure Blob Storage and a serverless function with Azurite, automate with GitHub Actions, and document two measured optimisations. Every Azure service is emulated locally; no real cloud resources are used.

100

marks: T1–T4 20 each, T5 5, video 10, contribution 5

4 / 5

technical tasks complete locally with real evidence

0

blockers; remaining: video, contribution report, dated screenshots (section 9)

1

green GitHub Actions run (#4, 1m 23s)

Source repository (for verification)

What	Link
Repository	https://github.com/YUANDONG-YANG/cloud-computing
Project folder (all code, tests, docs)	https://github.com/YUANDONG-YANG/cloud-computing/tree/main/Project1
CI/CD workflow file	https://github.com/YUANDONG-YANG/cloud-computing/blob/main/.github/workflows/deploy.yml
GitHub Actions (all runs)	https://github.com/YUANDONG-YANG/cloud-computing/actions
Green run #4 (Task 4 evidence)	https://github.com/YUANDONG-YANG/cloud-computing/actions/runs/36793291529
Commit history	https://github.com/YUANDONG-YANG/cloud-computing/commits/main
Contributors	https://github.com/YUANDONG-YANG/cloud-computing/graphs/contributors

The page you are reading is self-contained; these links let you check the same files, test run and commits online. Repository access may need to be granted to the reviewer if it is private.

Team

Member	Role in this document	Section presented in the video	Own contributions (commits, reviews, hours)
Yuandong Yang (GitHub: YUANDONG-YANG)	Team member, repository owner	to be assigned	to be filled with real data
Ethan Bayarsaikhan	Team member	to be assigned	to be filled with real data
Justin Norman-Rance	Team member	to be assigned	to be filled with real data

All three are contributors of this project. Task division, hours, meetings, commits and pull requests must be entered from real records (see the contribution report); nothing here is invented. Student IDs and the other members' GitHub accounts are not recorded yet.

1. Introduction

The assignment asks for a cloud-native application that processes and analyses the `All_Diets.csv` dataset and extracts nutritional insights (protein, carbs, fat) per diet type and cuisine, using Azure-style services, a NoSQL store, serverless processing, cloud storage and CI/CD. This page follows the order of the assignment document: Introduction, Prerequisites, Dataset overview, Tasks 1–5, Marking criteria, Submission requirements. For each task it shows the requirement, what was built, the screenshot evidence in the place where the assignment shows an empty screenshot box, and the status.

Result in one line: 7,806 recipes were analysed; results were produced in a Docker container, through an Azurite Blob and a manually invoked function into a JSON “NoSQL” store; 10 tests and a benchmark passed locally. The GitHub Actions pipeline ran green (run #4); the team video and the contribution evidence are still open.

2. Prerequisites and environment

Assignment prerequisite	How it was met
Start Project 1 during Labs 1 and 2; finish afterwards	All work ran in the Ubuntu VM prepared by the labs (system clock visible in every screenshot).
Preconfigured Ubuntu VM with Python, Docker, Pandas, Matplotlib, Seaborn, VS Code	Docker 29-era Compose stack, Python 3.11 images, pinned <code>requirements.txt</code> .
Additional simulation tools introduced later	Azurite 3.35.0 (Docker), Azure Blob SDK, local registry 2.8.3, pytest, flake8.
No real Azure resources	Only local emulators; the code refuses non-local endpoints.

3. Dataset overview

Columns: `Diet_type`, `Recipe_name`, `Cuisine_type`, `Protein(g)`, `Carbs(g)`, `Fat(g)`, `Extraction_day`, `Extraction_time`. Source: course resources; 702,514 bytes, 7,806 rows, 5 diet types (dash, keto, mediterranean, paleo, vegan); SHA-256 `0aa45c5e...f811`.

Data quality handled: mixed-case labels (“Paleo” and “paleo”) are merged; invalid or missing macros are filled with the column mean (none needed here); 38 recipes have zero fat, so their Carbs-to-Fat ratio is left undefined instead of infinity.

4. Tasks and instructions

Task 1 — Dataset analysis and insights (20 marks)

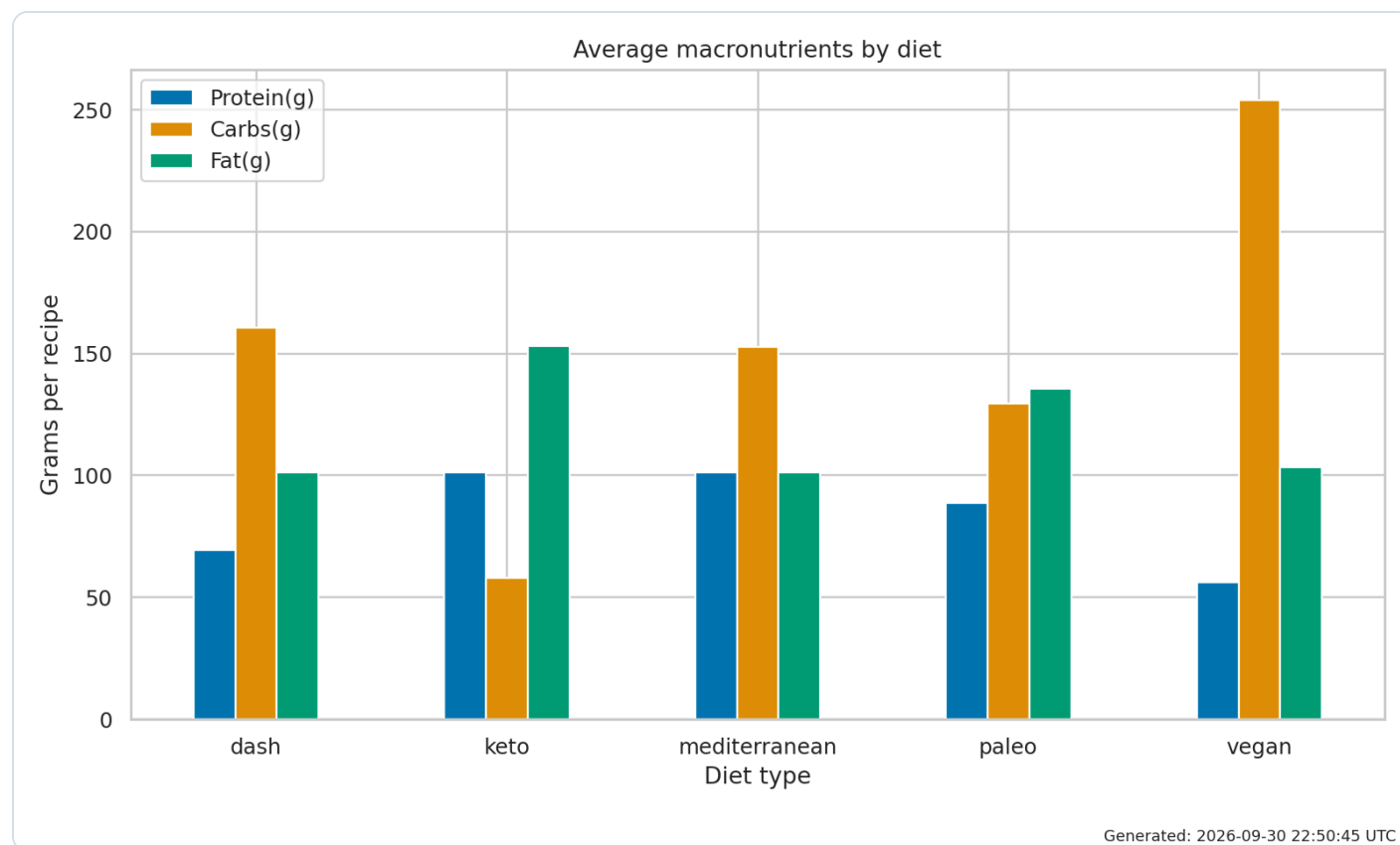
Assignment requirement	What was done	Status
Average protein, carbs, fat per diet type	groupby mean in nutrition.py; table below	done
Top 5 protein-rich recipes per diet type	Stable sort, 5 rows per diet (25 rows) in top5_protein_recipes.csv	done
Diet type with the highest protein	Keto on mean protein (101.27 g); mediterranean on total protein (177,249.89 g); both reported	done
Most common cuisines per diet type	common_cuisines.csv, ties preserved	done
Protein-to-carbs and carbs-to-fat ratios per recipe	Two new columns; zero denominators stay undefined	done
Clean missing values	Numeric coercion, invalid and missing → column mean (numeric columns only; the assignment's <code>df.fillna(df.mean())</code> would fail on text columns in current Pandas)	done
Bar charts, heatmaps, scatter plots (Matplotlib / Seaborn)	Three PNG charts, headless Agg backend, generation time printed on each	done
Deliverables: data_analysis.py, visualisations, screenshots with date and time	Script and charts below; terminal screenshot shows the VM clock	done

Screenshot box 1 — data processing, calculations (16:53:01)

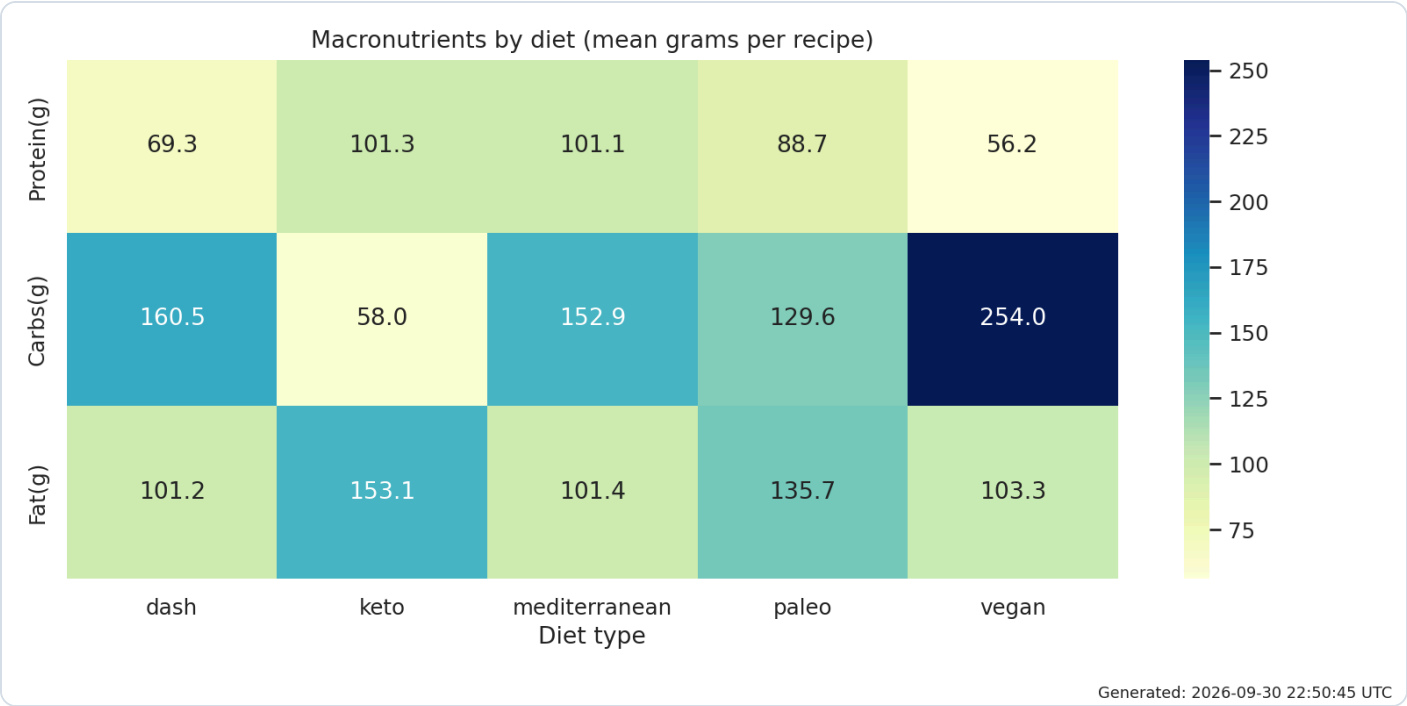
```
Activities Terminal Wed Sep 30 16:53:01
CPSY 300 Project 1 | Task 1 | Saved execution log and output
2026-09-30T16:52:59-06:00
Container nutritional-insights-analysis-run-e8bc1bc8a8ab Creating
Container nutritional-insights-analysis-run-e8bc1bc8a8ab Created
2026-09-30 22:50:45 UTC
Diet_type Protein(g) Carbs(g) Fat(g)
dash 69.28 160.54 101.15
keto 101.27 57.97 153.12
mediterranean 101.11 152.91 101.42
paleo 88.67 129.55 135.67
vegan 56.16 254.00 103.30
Most common cuisines:
Diet_type Cuisine_type count
dash american 639
keto american 663
mediterranean mediterranean 1274
paleo american 535
vegan american 925
{
  "generated_at": "2026-09-30 22:50:45 UTC",
  rows : 7806
  diet_types : 5
  highest_mean_protein_diets : ['keto']
  highest_mean_protein_g : 101.26650793650794
  highest_total_protein_diets : ['mediterranean']
  highest_total_protein_g : 177249.89
  undefined_ratios : {'Protein_to_Carbs_ratio': 0, 'Carbs_to_Fat_ratio': 38}
```

Averages per diet, most common cuisines, summary (7,806 rows, 5 diets, 38 undefined Carbs-to-Fat ratios).

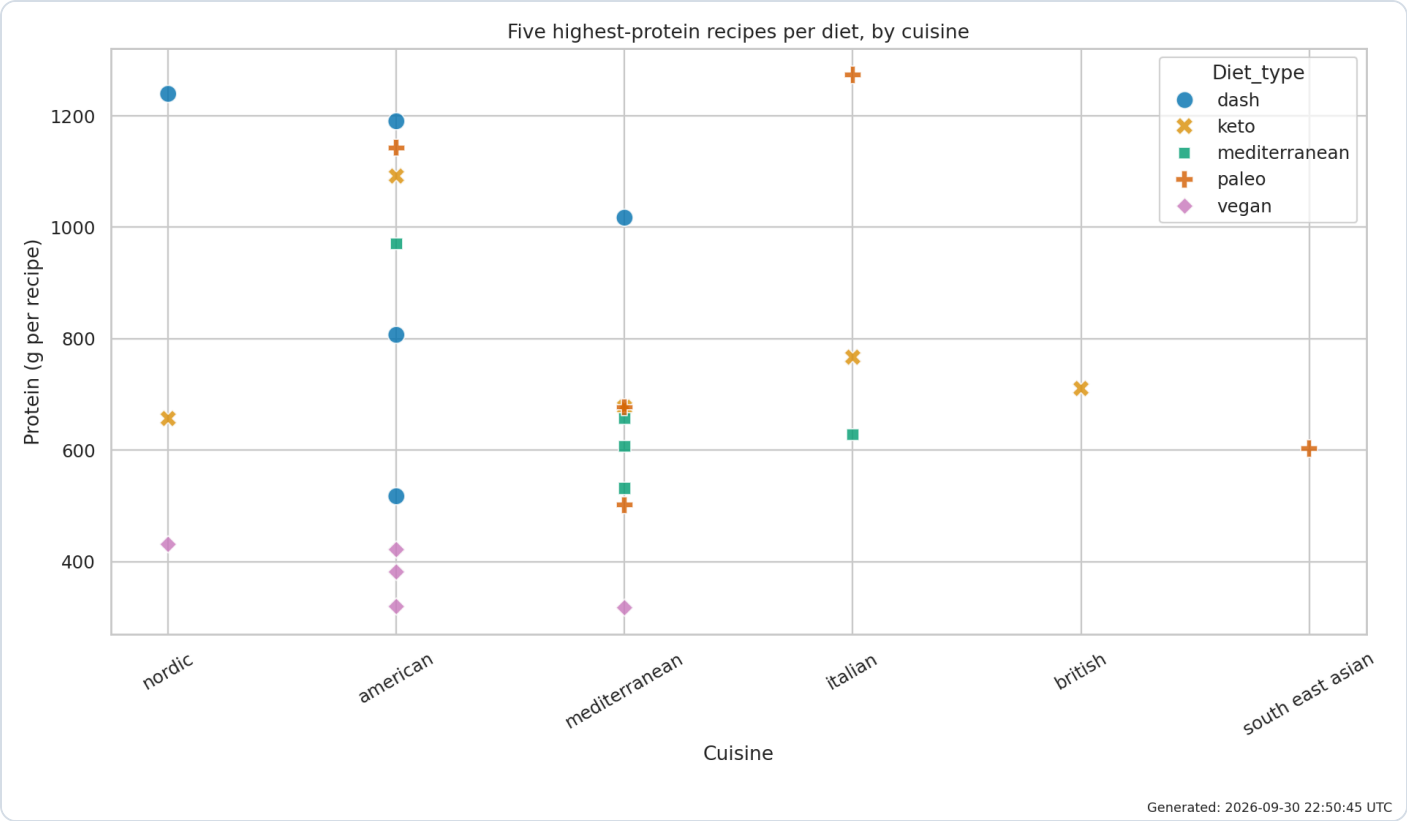
Visualisations



Bar chart: average protein, carbs and fat per diet.



Heatmap: macronutrient content by diet type.



Scatter: the 25 top protein recipes by cuisine.

Key results

Diet	Avg protein g	Avg carbs g	Avg fat g	Top cuisine	Total protein g
dash	69.28	160.54	101.15	american (639)	120,897.57
keto	101.27	57.97	153.12	american (663)	153,114.96
mediterranean	101.11	152.91	101.42	mediterranean (1,274)	177,249.89
paleo	88.67	129.55	135.67	american (535)	112,971.65
vegan	56.16	254.00	103.30	american (925)	85,471.00

Highest *mean* protein: keto. Highest *total* protein: mediterranean (depends on recipe count). Values are per recipe, not per serving.

Task 2 — Dockerizing the data processing application (20 marks)

Assignment requirement	What was done	Status
Write a Dockerfile (example: python:3.9-slim)	Multi-stage Dockerfile on python:3.11-slim: wheel builder stage, runtime without pip cache, non-root user (UID 1000)	done
<code>docker build -t diet-analysis .</code>	Built as diet-analysis:project1 (568 MB)	done
Run the container and verify processing	<code>docker compose run --rm analysis</code> ; outputs mounted to the host	done
Push to Docker Hub or a local registry (optional)	Local registry localhost:5001: push, pull, run; digest sha256:f5595b07.... Docker Hub not used	partly
Simulate deployment with Docker Compose or Minikube	Compose services azurite, analysis, upload, function, plus registry profile	done
Deliverables: Dockerfile; screenshots of run, push, Compose	Files in section 11; screenshot below	done

Screenshot box 2 — containers, images and registry (16:53:04)

```

CPSY 300 Project 1 | Task 2 | Live services and registry logs
2026-09-30T16:53:01-06:00
NAME                                IMAGE                                COMMAND                                SERVICE  CREATED        STATUS
nutritional-insights-azurite-1      mcr.microsoft.com/azure-storage/azurite:3.35.0  "docker-entrypoint.s..."  azurite   6 minutes ago  Up 6 m
inutes (healthy) 127.0.0.1:10000->10000/tcp
nutritional-insights-registry-1     registry:2.8.3                                "/entrypoint.sh /etc..."  registry  5 minutes ago  Up 5 m
inutes 127.0.0.1:5001->5000/tcp
Info -> U In Use

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
diet-analysis:baseline              91d9d3648d81                      698MB        193MB
diet-analysis:project1              f5595b07f723                      568MB        130MB  U
diet-analysis:tests                  0dd76a2c1444                      575MB        131MB

Saved docker push result:
6b37362b3da7: Pushed
2a8acfc0df46: Pushed
project1: digest: sha256:f5595b07f72304299087700bbc2459eed3fe069d3267db0c066a334b177ba8f1 size: 856

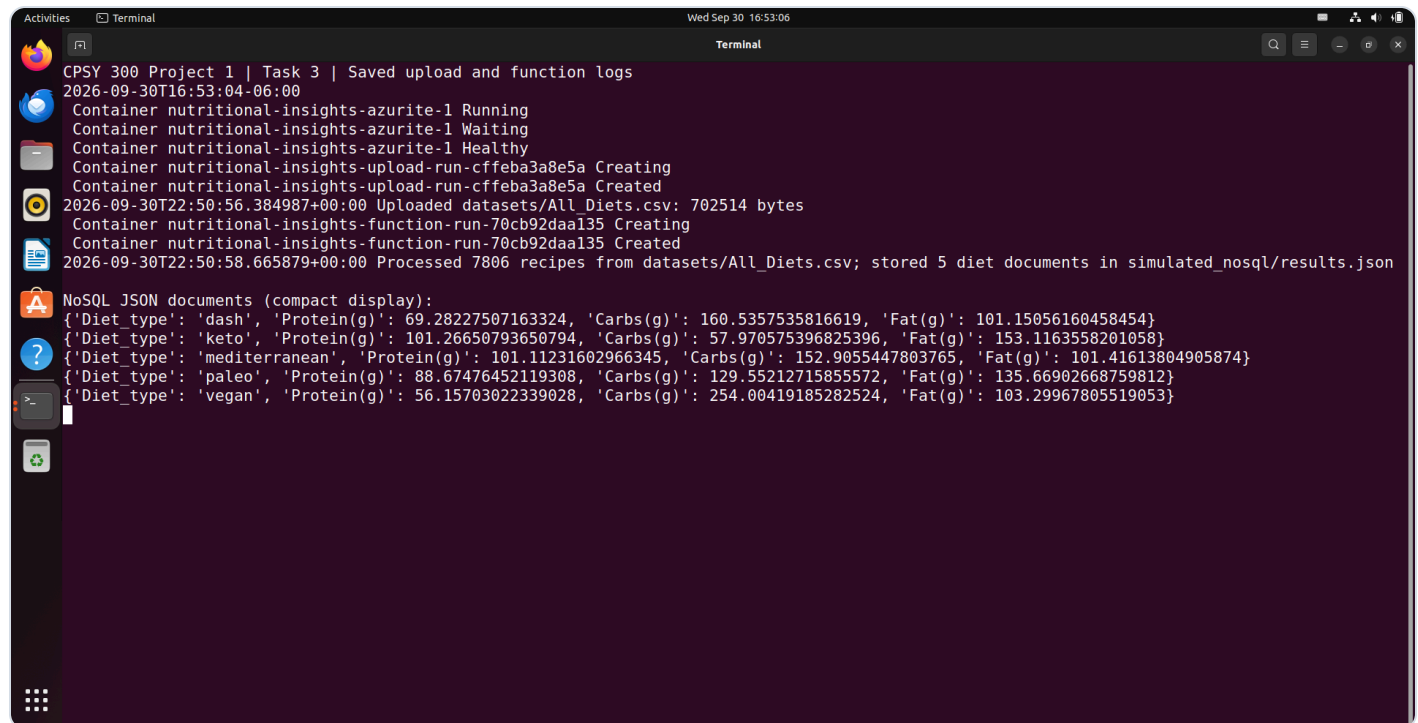
Saved docker pull result:
project1: Pulling from diet-analysis
Digest: sha256:f5595b07f72304299087700bbc2459eed3fe069d3267db0c066a334b177ba8f1
Status: Image is up to date for localhost:5001/diet-analysis:project1
localhost:5001/diet-analysis:project1
  
```

Running Compose services (Azurite healthy, registry on 127.0.0.1:5001), image sizes (baseline 698 MB, project1 568 MB), registry push and pull with the same digest.

Task 3 — Serverless processing with Azurite (20 marks)

Assignment requirement	What was done	Status
Install and start Azurite (npm or Docker)	Docker image azurite:3.35.0 as a health-checked Compose service, Blob port 10000	done
Upload All_Diets.csv with Azure Storage Explorer	Uploaded by script upload_dataset.py into container datasets; Storage Explorer itself was not used	partly
Function reads the CSV from Azurite with the Blob SDK	lambda_function.py uses BlobServiceClient against the emulator	done
Calculate average protein, carbs, fat per diet	Same shared module as Task 1; results verified equal to the batch path	done
Store results in a simulated NoSQL database (JSON or MongoDB)	Five documents in simulated_nosql/results.json, written atomically	done
Manual invocation or a file watcher	Manual invocation (docker compose run --rm --no-deps function)	done
Explain how cloud storage and the serverless flow were simulated	Azurite = Blob Storage, Python handler = serverless function, JSON file = NoSQL store; connection code only accepts local endpoints	done

Screenshot box 3 — Azurite running, CSV uploaded, function run, JSON result (16:53:06)

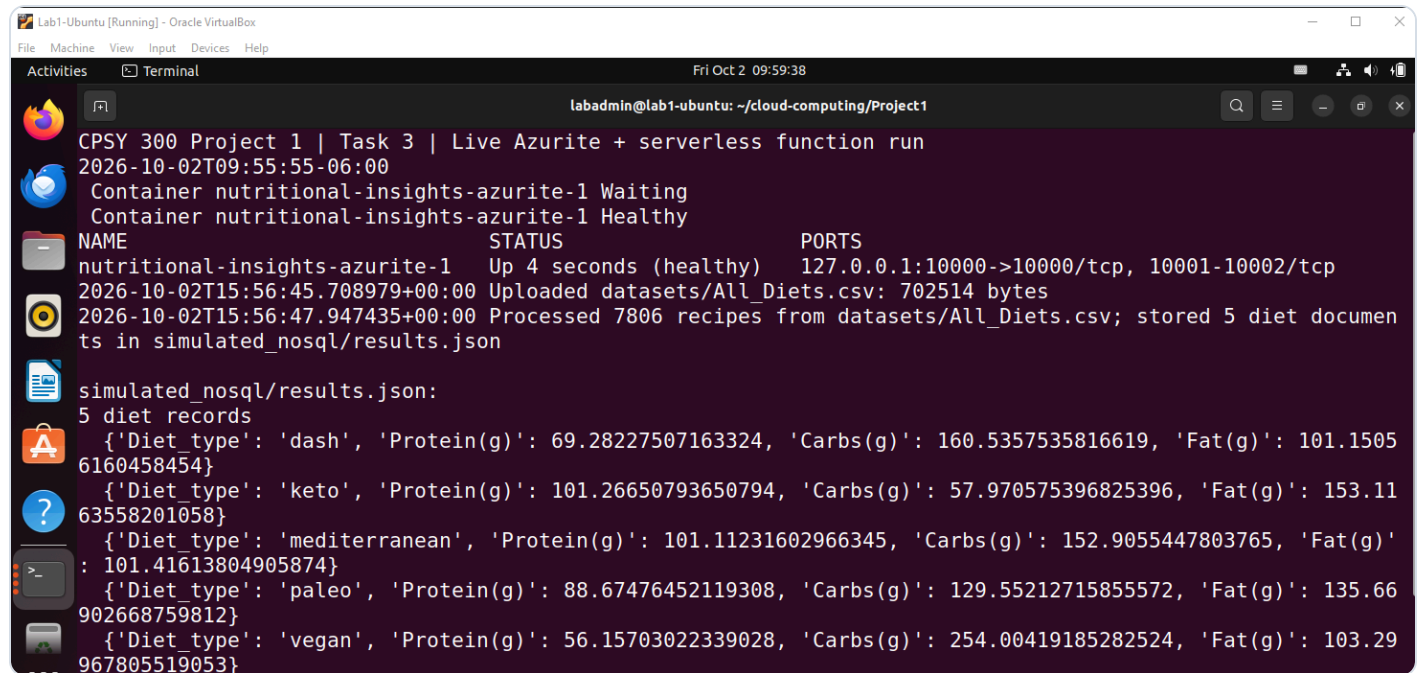


A screenshot of a terminal window titled 'Terminal' showing the execution of a function. The output includes status messages for containers, upload progress, and a list of 5 diet documents in JSON format.

```
CPSY 300 Project 1 | Task 3 | Saved upload and function logs
2026-09-30T16:53:04-06:00
Container nutritional-insights-azurite-1 Running
Container nutritional-insights-azurite-1 Waiting
Container nutritional-insights-azurite-1 Healthy
Container nutritional-insights-upload-run-cffeba3a8e5a Creating
Container nutritional-insights-upload-run-cffeba3a8e5a Created
2026-09-30T22:50:56.384987+00:00 Uploaded datasets/All Diets.csv: 702514 bytes
Container nutritional-insights-function-run-70cb92daal35 Creating
Container nutritional-insights-function-run-70cb92daal35 Created
2026-09-30T22:50:58.665879+00:00 Processed 7806 recipes from datasets/All_Diets.csv; stored 5 diet documents in simulated_nosql/results.json

NoSQL JSON documents (compact display):
{'Diet_type': 'dash', 'Protein(g)': 69.28227507163324, 'Carbs(g)': 160.5357535816619, 'Fat(g)': 101.15056160458454}
{'Diet_type': 'keto', 'Protein(g)': 101.26650793650794, 'Carbs(g)': 57.970575396825396, 'Fat(g)': 153.1163558201058}
{'Diet_type': 'mediterranean', 'Protein(g)': 101.11231602966345, 'Carbs(g)': 152.9055447803765, 'Fat(g)': 101.41613804905874}
{'Diet_type': 'paleo', 'Protein(g)': 88.67476452119308, 'Carbs(g)': 129.55212715855572, 'Fat(g)': 135.66902668759812}
{'Diet_type': 'vegan', 'Protein(g)': 56.15703022339028, 'Carbs(g)': 254.00419185282524, 'Fat(g)': 103.29967805519053}
```

Azurite healthy; CSV uploaded (702,514 bytes); function processed 7,806 recipes and stored 5 diet documents.



A screenshot of a terminal window titled 'Terminal' showing the execution of a function. The output includes status messages for containers, upload progress, and a list of 5 diet documents in JSON format.

```
CPSY 300 Project 1 | Task 3 | Live Azurite + serverless function run
2026-10-02T09:55:55-06:00
Container nutritional-insights-azurite-1 Waiting
Container nutritional-insights-azurite-1 Healthy
NAME                                STATUS                                PORTS
nutritional-insights-azurite-1      Up 4 seconds (healthy)               127.0.0.1:10000->10000/tcp, 10001-10002/tcp
2026-10-02T15:56:45.708979+00:00 Uploaded datasets/All Diets.csv: 702514 bytes
2026-10-02T15:56:47.947435+00:00 Processed 7806 recipes from datasets/All_Diets.csv; stored 5 diet documents in simulated_nosql/results.json

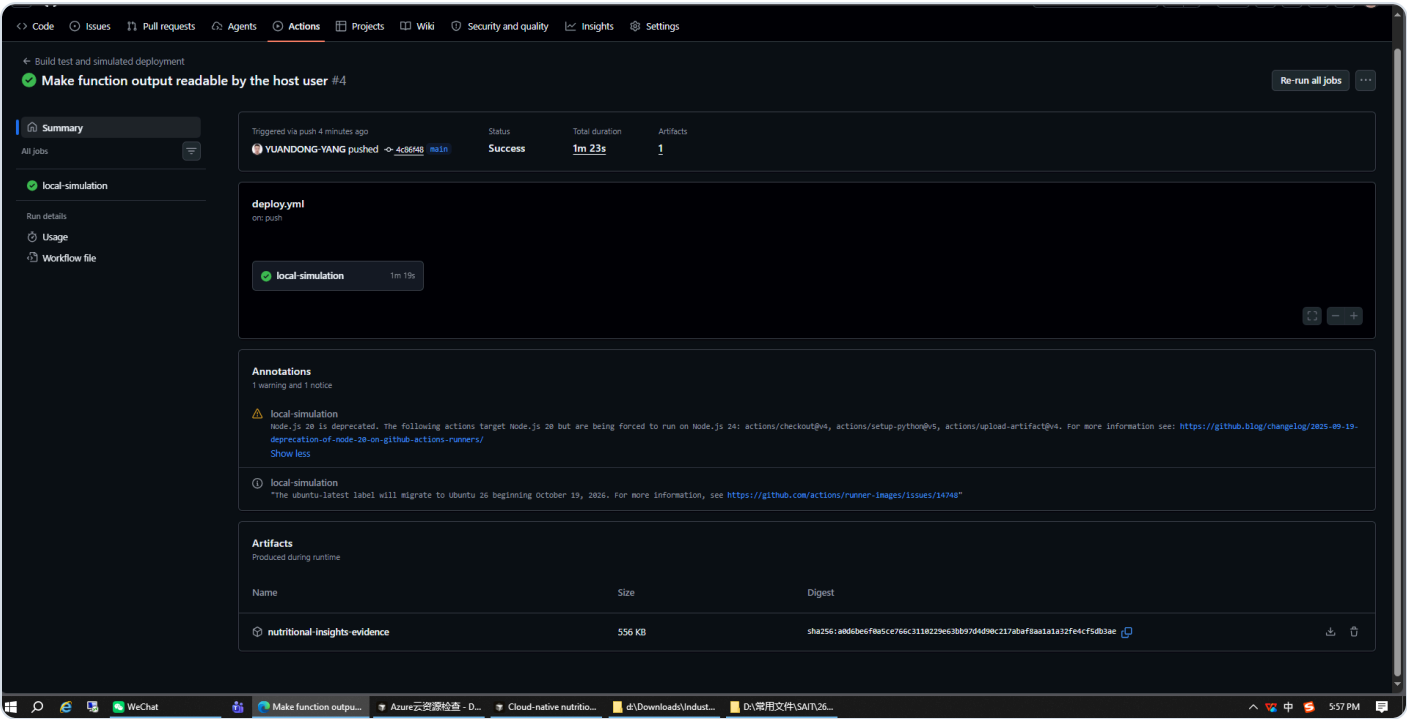
simulated_nosql/results.json:
5 diet records
{'Diet_type': 'dash', 'Protein(g)': 69.28227507163324, 'Carbs(g)': 160.5357535816619, 'Fat(g)': 101.15056160458454}
{'Diet_type': 'keto', 'Protein(g)': 101.26650793650794, 'Carbs(g)': 57.970575396825396, 'Fat(g)': 153.1163558201058}
{'Diet_type': 'mediterranean', 'Protein(g)': 101.11231602966345, 'Carbs(g)': 152.9055447803765, 'Fat(g)': 101.41613804905874}
{'Diet_type': 'paleo', 'Protein(g)': 88.67476452119308, 'Carbs(g)': 129.55212715855572, 'Fat(g)': 135.66902668759812}
{'Diet_type': 'vegan', 'Protein(g)': 56.15703022339028, 'Carbs(g)': 254.00419185282524, 'Fat(g)': 103.29967805519053}
```

Task 3 live run on the Ubuntu VM, Fri Oct 2 2026 09:59 (clock visible): Azurite healthy on 127.0.0.1:10000, CSV uploaded (702,514 bytes), function processed 7,806 recipes and stored 5 diet documents in simulated_nosql/results.json.

Task 4 — CI/CD pipeline with GitHub Actions (20 marks)

Assignment requirement	What was done	Status
Create a GitHub repository	Repository exists (folder Project1 in cloud-computing)	done
Workflow: build image, run tests, push to a registry	deploy.yml: build, pytest (10 tests), flake8, Azurite flow, output comparison, local registry push/pull/run, optional Docker Hub push with secrets	partly
Commit and push, monitor the run, confirm success	Run #4 on main succeeded in 1m 23s (job local-simulation 1m 19s); artifact nutritional-insights-evidence (556 KB) uploaded	done
Deliverables: .github/workflows/deploy.yml, run-log screenshots with date and time, deployment evidence	File is in section 11.1; GitHub run screenshot and local test evidence below	done

Screenshot box 4a — GitHub Actions run #4: Success (taskbar clock 5:57 PM)



Workflow “Build test and simulated deployment”, commit 4c86f48 on main: status Success, total duration 1m 23s, job local-simulation green (1m 19s), artifact nutritional-insights-evidence (556 KB). The two annotations are only Node.js and Ubuntu-label deprecation notices. Note: the screen shows the time (5:57 PM) but not the calendar date; the run was triggered on 2026-09-30 (see run page).

Screenshot box 4b — local tests and checks (16:53:09)

```

Activities  Terminal  Wed Sep 30 16:53:09
CPSY 300 Project 1 | Task 4 | LOCAL evidence; GitHub run pending
2026-09-30T16:53:07-06:00

Saved pytest output:
..... [100%]
10 passed in 0.71s

Saved successful flake8 + output verification:
PASS: batch and Azurite means match; all three charts exist

Measured aggregation comparison (median of 15 runs):
7806 rows: repeated filtering 13.722 ms -> groupby 1.245 ms; 11.02x
156120 rows: repeated filtering 90.752 ms -> groupby 11.964 ms; 7.59x

```

10 tests passed in 0.71 s; batch and Azurite means match; benchmark lines.

Task 5 — Search for enhancement (5 marks)

Assignment requirement	What was done			Status
Choose two enhancement options	(a) multi-stage Docker build, smaller image; (c) faster data processing			done
Research and apply to one of Tasks 1–4	Docker multi-stage documentation and Pandas GroupBy guide; applied to Task 2 (image) and Tasks 1/3 (aggregation)			done
One-page report: options, research, improvements, impact	One-page PDF with before/after numbers (text in section 11.5)			done
Measurement		Before	After	Change
Image size		697.66 MB	568.16 MB	–18.6 %
Aggregation, 7,806 rows (median of 15)		13.72 ms	1.25 ms	11.0× faster
Aggregation, 156,120 rows (20× replicated)		90.75 ms	11.96 ms	7.6× faster

Most of the size saving comes from excluding the full base image and pip cache, not from having two stages alone. The larger workload repeats the same rows and is a stress test only. Evidence: screenshot boxes 2 and 4, and benchmark.json in section 11.4.

5. Marking criteria (100 marks)

Item	Marks	Status	Note
Task 1 analysis and insights	20	done	All items and three chart types
Task 2 Docker	20	done	Local registry and Compose; Docker Hub optional
Task 3 Azurite and function	20	done	Upload by script, not Storage Explorer
Task 4 CI/CD	20	done	Workflow green on GitHub (run #4); add a dated screenshot
Task 5 enhancement report	5	done	One page, measured
Video presentation	10	not done	Guide and 11-slide deck prepared; recording needed
Contribution report	5	not done	Template only; needs real data

6. Submission requirements

Required item	Status	Where
data_analysis.py	done	Section 11.1
lambda_function.py	done	Section 11.1
Dockerfile	done	Section 11.1
.github/workflows/deploy.yml	partly	Section 11.1; must be pushed and run
PDF report with all screenshots and the 1-page enhancement report, date and time visible	partly	PDFs in section 11.5; add the GitHub Actions screenshots
Team video, every face visible	not done	Guide in section 11.5
Contribution report (tasks, commits, PRs, attendance, hours, agreements, communication)	not done	Template in section 11.5

7. Notes on interpretation

Values are per recipe, not per serving; no medical conclusions are drawn.

8. Architecture

```
All_Diets.csv
├─ data_analysis.py → output/ (tables, 3 charts, summary.json)      [Task 1, Docker:]
└─ upload_dataset.py → Azurite Blob "datasets/All_Diets.csv"        [Task 3]
    └─ lambda_function.py (manual) → simulated_nosql/results.json
GitHub push → Actions: build → pytest + flake8 → Azurite flow → verify_outputs
                → local registry push/pull → run pulled image → upload evidence [Task 4]
Shared logic: nutrition.py (used by batch and function, so results must match)
```

9. Review findings and their status

Non-English content removed from the repository resolved

The project rule is that no Chinese text may appear on Git. The working notes file and a Chinese-named readme were deleted from the tree, and two recipe names in data/All_Diets.csv that contained non-Latin characters were cleaned (recipe-name text only; all means are identical, verified). See data/README.md. Commit messages and documents are English. **Note:** earlier commits in Git history still contain the removed files; rewriting history was deliberately not done without the owner's decision.

One consistent code version resolved

The repository is the single source of truth. Its code, logs, screenshots and benchmark numbers belong together (10 tests pass, flake8 is clean, results reproduce). The earlier divergent loose copy of the code was not committed.

Repository layout resolved

Required files stay at Project1/ root (data_analysis.py, lambda_function.py, Dockerfile); results, evidence and reports live under docs/; generated output is ignored; the workflow sits at the repository root with working-directory: Project1, which is where GitHub reads it.

GitHub Actions pipeline green resolved

Two earlier runs failed for real reasons: generated files had been committed and were not writable by the container user, and the function wrote `results.json` with mode 0600 so the runner could not read it. Both were fixed (untracked outputs; mode 0644). Run #4 succeeded: build, tests and lint, Azurite flow, output comparison, local registry push/pull/run, artifact upload. Remaining improvement: a screenshot that also shows the calendar date. Docker Hub publishing stays off until `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` are configured.

Honest scope notes keep in report and video

- The function is a manually invoked script, not an Azure Functions host; Azurite offers no event triggers. The NoSQL store is a JSON file; the registry is local.
- Upload was scripted rather than done through Azure Storage Explorer.
- The benchmark covers aggregation only; the 20x workload repeats the same rows. Most of the image-size saving comes from dropping the full base image and pip cache.

TODO for teammates (Ethan Bayarsaikhan, Justin Norman-Rance)

Code, Tasks 1-5 and the PDF report are done by Yuandong. The items below need real input from each member. Do not invent records.

- **[] Contribution Report (5 marks, required):** open `docs/reports/Contribution-Report.html` and send Yuandong, for each member: student ID, tasks done, hours spent, meetings attended (date, duration), commit/PR/review links. Fields in [brackets] are empty.
- **[] GitHub activity (Ethan, Justin):** accept the repository invitation, then make at least one real commit or pull request of your own work and review one another's changes. All 7 current commits are by Yuandong, so collaboration evidence is missing.
- **[] Meeting and communication evidence:** screenshot Teams chat or calendar showing meetings and task division; state how often the team communicated.
- **[] Video presentation (10 marks, required):** every member appears on camera and presents their own part (see `Video-Guide.html` and the 11-slide deck, about 8 minutes). Agree who presents which Task: _____.
- **[] Task 3 screenshot (Yuandong or whoever has the VM):** install Azure Storage Explorer, connect to Azurite, upload `All_Diets.csv` to container datasets, and capture a screenshot with date and time.
- **[] Optional, Task 2/4:** push the image to Docker Hub (set `DOCKERHUB_USERNAME` variable and `DOCKERHUB_TOKEN` secret) and screenshot it.
- **[] Final step (Yuandong):** after the above, re-export the PDF and zip the submission.

10. Action checklist before submission

1. **Clean Git:** done for the current tree; history still holds the removed files.
2. **Pick one code version:** done; the repository is the source of truth.
3. **Workflow:** located at the repository root; green run #4 captured (add a version that also shows the calendar date).
4. **Optional Docker Hub:** set DOCKERHUB_USERNAME variable and DOCKERHUB_TOKEN secret; screenshot the pushed tag.
5. **Real team evidence:** each member commits and reviews their own changes; fill names, IDs, hours, meetings, links into the contribution report; do not fabricate history.
6. **Record the video** with everyone visible, following the slide deck (11 slides, ~8 minutes).
7. **Assemble the zip:** data_analysis.py, lambda_function.py, Dockerfile, .github/workflows/deploy.yml, the PDF (screenshots + 1-page enhancement report), the video, the contribution report. Exclude stale duplicates.

11. Complete deliverables (embedded, open any item)

Everything produced for the project is included below so this single file can be reviewed offline. Click a heading to expand. The course dataset itself is not embedded; its provenance is recorded in 10.4.

11.1 Required submission files

- ▶ data_analysis.py required by the assignment
- ▶ lambda_function.py required by the assignment
- ▶ Dockerfile required by the assignment
- ▶ .github/workflows/deploy.yml required by the assignment

11.2 Supporting source code

- ▶ nutrition.py
- ▶ upload_dataset.py
- ▶ storage_config.py
- ▶ docker-compose.yml

► Dockerfile.baseline

► Dockerfile.test

► tests/test_nutrition.py

► scripts/verify_outputs.py

► scripts/benchmark.py

► requirements.txt

► requirements-dev.txt

► pytest.ini

► .flake8

► .dockerignore

► README.md

11.3 Result tables (Task 1 and Task 3)

Average macronutrients per diet (g per recipe)

Diet_type	Protein(g)	Carbs(g)	Fat(g)
dash	69.28	160.54	101.15
keto	101.27	57.97	153.12
mediterranean	101.11	152.91	101.42
paleo	88.67	129.55	135.67
vegan	56.16	254.00	103.30

Total protein per diet (g)

Diet_type	Protein(g)
dash	120,897.57
keto	153,114.96
mediterranean	177,249.89
paleo	112,971.65
vegan	85,471.00

Most common cuisine per diet

Diet_type	Cuisine_type	count
dash	american	639
keto	american	663
mediterranean	mediterranean	1274
paleo	american	535
vegan	american	925

Top 5 protein-rich recipes per diet (25 rows, with both new ratios)

Diet_type	Recipe_name	Cuisine_type	Protein(g)	Carbs(g)	Fat(g)	Extraction_day
dash	Salmon Mousse	nordic	1,239.47	22.40	983.36	10/16/2022
dash	Homemade Turkey Alphabet Soup	american	1,190.35	49.13	297.45	10/16/2022
dash	Barbecue Chicken Legs	mediterranean	1,017.25	241.93	1,002.07	10/16/2022
dash	12th Man Hot Wings	american	807.03	49.17	649.49	10/16/2022
dash	Jazzed Up Turkey Tetrazzini	american	517.47	575.58	377.75	10/16/2022
keto	Sara Louise's Keto Smoked Holiday Turkey	american	1,092.00	49.28	313.81	10/16/2022
keto	Mayo Free Deviled Eggs (Paleo, Whole30 + Keto)	italian	766.99	41.29	287.59	10/16/2022
keto	Low Carb Beef and Cheddar Cauliflower Bake, THM S meal, Keto too	british	710.81	112.56	1,154.61	10/16/2022
keto	Orange and Five-Spice Roasted Chicken Legs Keto, Paleo	mediterranean	677.22	41.71	686.90	10/16/2022
keto	Low Carb Salmon Cakes	nordic	656.95	13.25	501.19	10/16/2022

	(Keto, Grain Free)					
mediterranean	Fava Bean Salad with Mountain Ham and Mint	american	970.31	823.34	322.84	10/16/2022
mediterranean	Poached Salt Cod with Vegetables and Garlic Mayonnaise	mediterranean	656.65	315.52	73.73	10/16/2022
mediterranean	Mediterranean Pizza	italian	628.29	3,405.55	333.76	10/16/2022
mediterranean	Orange and Cumin Leg of Lamb with Roasted Tomatoes and Garlic Recipe	mediterranean	607.03	109.27	489.93	10/16/2022
mediterranean	Meatballs Braised with Kale	mediterranean	531.48	279.16	566.41	10/16/2022
paleo	Swiss Paleo's Homemade Italian & Chorizo Sausage	italian	1,273.61	83.34	1,608.54	10/16/2022
paleo	Turkey Soup	american	1,142.58	693.64	287.86	10/16/2022
paleo	Orange and Five-Spice Roasted Chicken Legs Keto, Paleo	mediterranean	677.22	41.71	686.90	10/16/2022
paleo	Vietnamese Pho Pressure Cooker (Noodle Soup)	south east asian	602.91	274.87	400.01	10/16/2022

paleo	Paleo Slow Cooker Honey Garlic Chicken and Vegetables recipes	mediterranean	501.65	312.39	58.06	10/16/2022
vegan	Tangy Teriyaki Salmon	nordic	431.10	39.47	276.51	10/16/2022
vegan	Mini nut roasts with candied carrots	american	421.72	1,636.67	267.25	10/16/2022
vegan	Vegan Pinto Bean Chili	american	381.55	1,264.46	37.23	10/16/2022
vegan	Vegan Chili and Cornbread Casserole	american	319.54	1,159.00	291.70	10/16/2022
vegan	Kentucky fried seitan	mediterranean	317.43	703.99	534.41	10/16/2022

► `output/summary.json`

► `simulated_nosql/results.json` (simulated NoSQL store written by the function)

11.4 Logs and measurements

► `evidence/analysis.log`

► `evidence/upload.log`

► `evidence/function.log`

► `evidence/compose.log`

► `evidence/registry-push.log`

► `evidence/registry-pull.log`

▶ evidence/tests.log

▶ evidence/verification.log

▶ evidence/benchmark.log

▶ evidence/benchmark.json

▶ evidence/image-inspect.json

▶ evidence/installed-packages.txt

▶ evidence/dataset-provenance.json

11.5 Reports

▶ Task 5 one-page enhancement report (text)

PDF: [Evidence report, 10 pages \(all screenshots\)](#). (Project1-Report.pdf, embedded — click to open or save)

PDF: [Task 5 enhancement report, 1 page](#) (Project1-Enhancement-Report.pdf, embedded — click to open or save)

HTML: [Video demonstration guide](#)

HTML: [Contribution report template \(to be completed with real data\)](#).

Generated 2026-09-30 from a review of the project folder and the packaged zip. Pure-ASCII/English document suitable for Git.